

ELECTRONICS CLUB, IIT GUWAHATI

Digital Project

Lenna

FPGA-Based Real-Time Image Processing System

Jesswin George

Contents

1	Project Overview	2
1.1	Objective	2
1.2	Overall System Architecture	2
1.3	Performance and Throughput Analysis	3
2	Hardware Module Details	4
2.1	1D Horizontal Line Buffer (<code>line_buffer_param</code>)	4
2.2	2D Sliding Window Generator (<code>image_control_param</code>)	4
2.3	Convolution Engine (<code>conv3x3</code>)	4
2.3.1	Pipeline Architecture	4
2.4	Kernel Configuration (<code>kernel_selector</code>)	5
2.5	Helper Modules (Data Formatting & Conversion)	6
2.5.1	RGB to Grayscale Converter (<code>rgb2gray</code>)	6
2.5.2	Grayscale to RGB Converter (<code>gray2rgb</code>)	6
2.5.3	Sobel Edge Formatter (<code>sobel_edge_stream_rgb</code>)	6
3	System Integration and Control	7
3.1	Image Processing Wrapper (<code>image_process_wrapper</code>)	7
3.1.1	Operational Modes and Datapath Routing	7
3.1.2	Flow Control & AXI-Stream Backpressure	8
3.2	Top-Level System Architecture (<code>image_ip</code>)	8
3.2.1	Camera Acquisition Interface (<code>cam_top</code>)	8
3.2.2	Datapath Expansion and Truncation	9
3.2.3	Asynchronous Frame Buffering (Dual-Port BRAM)	9
4	Conclusion	10

Chapter 1

Project Overview

1.1 Objective

The primary objective of this project is to develop a high-performance hardware accelerator for real-time image filtering on an FPGA. To overcome the latency and throughput limitations of sequential software processing, this project employs a **fully pipelined streaming architecture**. By exploiting spatial parallelism, the system continuously processes pixels through dedicated hardware modules, achieving minimal latency and high throughput for complex digital signal processing (DSP).

Implemented on the **Nexys A7-100T FPGA** using an **OV7670 CMOS camera**, the system executes the entire pipeline from frame capture to external visualization. Key functionalities include:

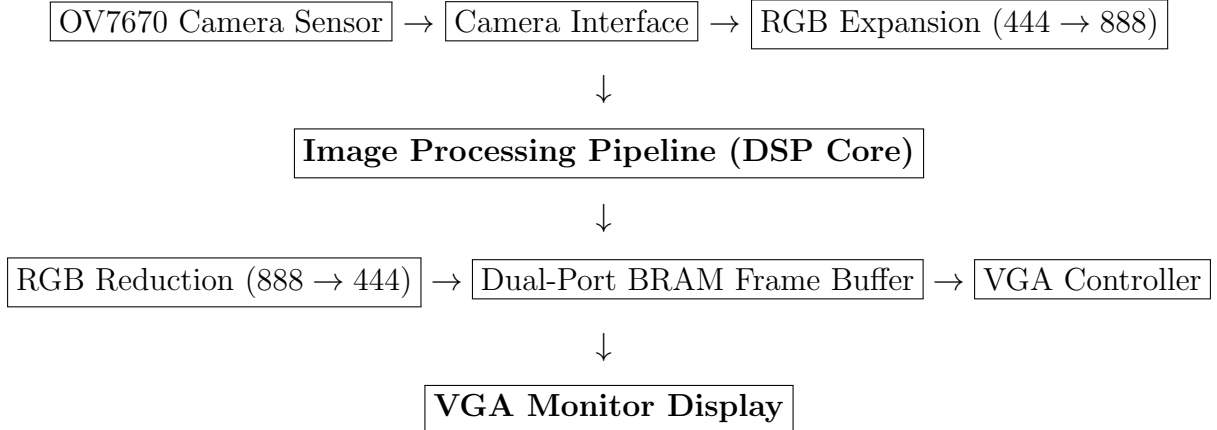
- **Real-time Acquisition:** Synchronous, continuous frame capture from the OV7670 camera.
- **Spatial Neighborhood Generation:** Hardware-based 3×3 sliding pixel windows using line buffers for spatial filtering.
- **Parallel Convolution:** High-speed execution of spatial filters (Sobel edge detection, Gaussian blur, sharpening) via parallel arithmetic.
- **Clock Domain Synchronization:** Reliable data transfer between the camera and VGA clock domains using dual-port BRAM.
- **Display Output:** Direct real-time visualization via a custom VGA controller.

This hardware-centric approach minimizes CPU intervention, making it highly effective for embedded vision systems and high-speed signal processing applications.

1.2 Overall System Architecture

The architecture is a unidirectional streaming pipeline where pixel data flows sequentially through specialized modules. This pipelined structure enables simultaneous multi-stage pixel processing to maximize throughput while seamlessly handling clock domain transitions and bit-depth conversions.

High-Level Data Flow



Raw pixels from the camera are formatted and processed by the DSP pipeline in real time. The filtered data is then buffered in dual-port RAM and fetched by the VGA controller for display.

1.3 Performance and Throughput Analysis

While software-based image processing struggles with the massive data rates of live video streams, this hardware architecture guarantees real-time performance through precise timing characteristics:

- **Sustained Throughput:** After an initial multi-row buffering delay to populate the sliding windows, the parallel DSP multipliers and adder trees guarantee a sustained throughput of exactly one processed pixel per clock cycle.
- **Real-Time Processing Margin:** Operating on the 25 MHz camera pixel clock, the DSP pipeline effortlessly processes 25 million pixels per second. This comfortably exceeds the ≈ 18.4 million pixels/second threshold required to sustain a 640×480 resolution at 60 Frames Per Second (FPS) without dropping frames or tearing.

Chapter 2

Hardware Module Details

2.1 1D Horizontal Line Buffer (`line_buffer_param`)

This module stores a single image row to act as the memory primitive for spatial filtering.

- **Output:** Generates a continuous 1×3 pixel window on every read cycle.
- **Addressing:** Uses $\lceil \log_2(\text{IMG_WIDTH}) \rceil$ for dynamic pointer sizing, ensuring optimal logic utilization.
- **Zero-Padding:** Look-ahead pointers automatically output zeros at image boundaries to prevent hardware artifacting.

2.2 2D Sliding Window Generator (`image_control_param`)

This module scales the 1D buffer to transform a linear pixel stream into a parallel 3×3 grid.

- **Allocation:** Instantiates four independent `line_buffer_param` modules.
- **Circular Buffering:** A 2-state FSM writes incoming data to one target buffer while concurrently reading the other three.
- **Stall-Free Output:** This "ping-pong" rotation guarantees a valid 3×3 window is generated every clock cycle without pipeline stalls.

2.3 Convolution Engine (`conv3x3`)

The `conv3x3` module is the mathematical core of the pipeline, executing 2D convolutions. It ingests a 72-bit packed 3×3 window and computes one spatially filtered output pixel per clock cycle.

2.3.1 Pipeline Architecture

To achieve real-time high-throughput processing, the convolution math is divided into three parallel hardware stages:

- **Stage 1: Parallel Multiplication**

Nine discrete DSP multipliers operate simultaneously, calculating the product of zero-extended pixels and their signed kernel coefficients: $mult[i] = p[i] \times k[i]$.

- **Stage 2: Adder Tree Accumulation**

A balanced combinational adder tree accumulates the nine multiplication results. This replaces a slow sequential cascade, drastically reducing critical path delay and maximizing operating frequency (F_{max}).

- **Stage 3: Normalization & Clamping**

The accumulated sum is divided by the `norm_factor` to maintain brightness. Combinational clamping logic then bounds the output strictly between 0 and 255, preventing overflow/underflow distortion.

2.4 Kernel Configuration (`kernel_selector`)

The `kernel_selector` is a combinational routing unit that supplies predefined 3×3 matrices and their normalization factors to the convolution engine. Controlled by a 7-bit priority-encoded signal, it allows dynamic switching of spatial filters without external memory reconfiguration.

Supported kernels include:

- **Identity:** Passes the pixel window unmodified.

$$K_{ID} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \text{Norm} = 1$$

- **Sobel X & Y (Edge Detection):** Approximates the image gradient. The system outputs both K_{SobelX} and K_{SobelY} simultaneously to separate color-channel engines to calculate overall edge magnitude.

$$K_{SobelX} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_{SobelY} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}, \quad \text{Norm} = 1$$

- **Sharpen:** Enhances high-frequency spatial components.

$$K_{Sharp} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}, \quad \text{Norm} = 1$$

- **Box Blur:** A normalized spatial low-pass (mean) filter.

$$K_{Box} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad \text{Norm} = 9$$

- **Gaussian Blur:** A weighted low-pass filter providing natural smoothing.

$$K_{Gauss} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}, \quad \text{Norm} = 16$$

2.5 Helper Modules (Data Formatting & Conversion)

These 1-cycle latency modules handle bit-width conversion, color mapping, and data formatting between the DSP core and external interfaces.

2.5.1 RGB to Grayscale Converter (rgb2gray)

Converts a 24-bit RGB pixel into an 8-bit grayscale pixel using a hardware-efficient luminance approximation, weighted to mimic human vision sensitivity.

- **Mathematical Approximation:** $Gray = \frac{R+2G+B}{4}$
- **Hardware Implementation:** Avoids DSP multipliers by utilizing standard addition and logical bit-shifts:

$$Gray_{out} = (R + (G \ll 1) + B) \gg 2$$

- **Effective Weights:** Red (0.25), Green (0.50), Blue (0.25).

2.5.2 Grayscale to RGB Converter (gray2rgb)

Handles bit-depth expansion for video interfaces (e.g., VGA) requiring 24-bit RGB vectors for monochrome images.

- **Operation:** When `in_valid` asserts, the 8-bit grayscale value replicates across the Red, Green, and Blue channels.
- **Output Logic:** $RGB_{out} = \{Gray, Gray, Gray\}$. (e.g., `8'h7F` becomes `{7F, 7F, 7F}`).

2.5.3 Sobel Edge Formatter (sobel_edge_stream_rgb)

Processes the raw horizontal (X) and vertical (Y) gradients to produce a binarized, high-contrast RGB edge map.

- **Magnitude Approximation:** Approximates magnitude using absolute sums to avoid hardware-intensive square roots: $Magnitude = |Sobel_X| + |Sobel_Y|$. The accumulation wire widens to `DATA_W+1` bits to prevent overflow.
- **Thresholding:** Compares the magnitude against a parameterized threshold (default `THRESHOLD = 80`).
- **Binarized RGB Output:**

$$Edge_{val} = (Magnitude > THRESHOLD) ? 255 : 0$$

Packs the result into a 24-bit vector ($Out_{pixel} = \{Edge_{val}, Edge_{val}, Edge_{val}\}$), ensuring pure white edges and black backgrounds.

Chapter 3

System Integration and Control

3.1 Image Processing Wrapper (`image_process_wrapper`)

Serving as the central processing hub, the `image_process_wrapper` dynamically routes the streaming 24-bit RGB data through its mathematical stages. By instantiating three independent spatial window generators and three convolution engines (one per color channel), the wrapper intelligently reconfigures its internal datapath to maximize hardware utilization based on the selected mode.

3.1.1 Operational Modes and Datapath Routing

The system shifts data across the available arithmetic units to achieve different visual effects without requiring redundant hardware blocks.

Mode A: Standard RGB Filtering

In this default state, the system processes full-color images:

- **Split:** The incoming 24-bit pixel divides into 8-bit R, G, and B components.
- **Process:** The Kernel Selector (`KER`) outputs a single filter (e.g., Gaussian Blur). All three convolution engines (`CONV_R`, `CONV_G`, `CONV_B`) independently apply this identical kernel.
- **Merge:** Outputs are concatenated back into a single 24-bit RGB vector.

Mode B: Grayscale Conversion

To conserve dynamic power during monochrome processing:

- **Luminance:** The `rgb2gray` module calculates the 8-bit intensity of the RGB pixel.
- **Process:** This single value is routed exclusively to the Red channel (`CONV_R`).
- **Broadcast:** A dedicated `gray2rgb` module duplicates the 8-bit result across all three channels for standard 24-bit video output.

Mode C: Sobel Edge Detection (Hardware Repurposing)

Calculating a spatial gradient typically requires two independent MAC arrays. To optimize logic, the wrapper repurposes the RGB datapath:

- **Routing:** The input is forced into grayscale and fed simultaneously into both the Blue and Green window generators.
- **Execution:** CONV_B computes the horizontal gradient (K_{SobelX}), while CONV_G simultaneously computes the vertical gradient (K_{SobelY}).
- **Magnitude:** A custom `sobel_edge_stream_rgb` block computes the unified orthogonal magnitude and applies a static threshold (e.g., 80) to generate a high-contrast edge map.

Image Inversion (Negative Mode)

A hardware-level color inversion feature with zero added pipeline latency. When the `negative` control flag is asserted, the final clamping logic utilizes a basic subtractor to calculate the complement of the processed pixel ($255 - \text{pixel}$).

3.1.2 Flow Control & AXI-Stream Backpressure

To prevent pipeline stalls and dropped frames, the wrapper implements an AXI-Stream compliant backpressure protocol using a synchronous FIFO:

- **Output Handshake (`m_axis_tvalid`):** Asserts when the core has valid processed data. It halts the pipeline if the downstream memory asserts that `m_in_ready` is low.
- **Input Backpressure (`s_out_ready`):** If the processing engine's internal queues approach capacity, `axis_prog_full` asserts. This is inverted and sent upstream, signaling the camera/DMA controller to pause transmission until the DSP core resolves the bottleneck.

3.2 Top-Level System Architecture (`image_ip`)

The `image_ip` module orchestrates the physical flow of pixels from the optical sensor to the display monitor. It manages the external peripherals, strictly controls Clock Domain Crossing (CDC), and handles asynchronous frame buffering.

3.2.1 Camera Acquisition Interface (`cam_top`)

The system captures live video from an external CMOS image sensor (e.g., the OV7670) via two main phases:

1. **Initialization:** The camera's internal DSP registers are configured via an I^2C bus (`sioc`, `siod`).
2. **Capture:** The module receives raw, byte-wise pixel data (`i_top_pix.byte`) strictly synchronized to the camera's discrete pixel clock (`i_top_pclk`), utilizing `VSNC` (start of frame) and `HREF` (line valid) signals to unpack the bytes into a synchronized 12-bit RGB444 format.

3.2.2 Datapath Expansion and Truncation

The convolution engines require high-precision 24-bit data to maintain arithmetic accuracy, necessitating dynamic width conversion:

- **Expansion (RGB444 $\rightarrow$ RGB888):** The 12-bit camera output is upsampled by duplicating the 4-bit channel values (e.g., a Red value of 1010_2 becomes 10101010_2).
- **Truncation (RGB888 $\rightarrow$ RGB444):** Post-convolution, the 24-bit data is truncated back to 12 bits by discarding the lower 4 bits of each channel. This strategic downscaling effectively halves the required Block RAM footprint with minimal loss of visual fidelity.

3.2.3 Asynchronous Frame Buffering (Dual-Port BRAM)

A True Dual-Port Block RAM (`mem_bram`) acts as an elastic buffer, bridging the bursty camera pipeline and the strict continuous-read timing of the VGA monitor.

Write Port (Port A) - Pipeline Synchronization:

Clocked by the camera's `i_top_pclk`. To resolve the latency of the 3×3 window generators, memory addressing ignores spatial camera coordinates. Instead, the write pointer (`wr_addr`) resets solely on the `VSYNC` frame anchor and increments strictly on `proc_out_valid`, cleanly mapping the 2D image into a sequential 1D array.

Read Port (Port B) - VGA Display (`vga_top`):

Clocked independently at 25 MHz (`w_clk25m`). The controller generates standard 640×480 @ 60Hz sync pulses and dynamically calculates the read address (`o_bram_pix_addr`), fetching 12-bit pixels exactly when the monitor's raster beam requires them.

Chapter 4

Conclusion

This project successfully implements a high-throughput, real-time image processing accelerator on an FPGA. By migrating spatial filters from sequential software to a fully pipelined hardware architecture, the system achieves massive parallelism and real-time video processing.

The design effectively resolves complex DSP challenges through strategic architectural choices. The use of circular line buffers ensures a continuous data stream, while balanced adder trees maximize the system’s operating frequency. Furthermore, the integration proves highly robust, successfully managing asynchronous clock domain crossings via dual-port BRAM and maintaining pipeline stability through AXI-Stream backpressure protocols.

Ultimately, this project demonstrates that optimized hardware design—such as repurposing RGB datapaths for gradient calculations—can significantly reduce logic footprint without sacrificing performance. This architecture provides a scalable and efficient foundation for advanced embedded vision and real-time video analytics.